PRODUCT REQUIREMENTS DOCUMENT

SignaSense

Emotion-Aware Sign Language Translator for iOS

Document Version	1.1 — Updated project name
Product Type	AI-powered iOS accessibility application
Platform	Apple iOS
Primary Language	Swift
Development Environment	Xcode
Target Duration	24-hour hackathon MVP
Target Sign Language	Indian Sign Language (ISL), subject to dataset validation
Status	Proposed

Product vision: Make everyday communication more accessible by connecting signed gestures, written language, and spoken expression through mobile technology.

Prepared for product planning, hackathon pitching, and implementation.

Table of Contents

1. Executive Summary
2. Problem Statement
3. Product Objectives
4. Target Users and Use Cases
5. Product Scope
6. Functional Requirements
7. Detailed Technology Stack
8. System Architecture
9. Recommended Application Structure
10. Data and Model Requirements
11. User Experience and Interface Requirements
12. Non-Functional Requirements
13. Privacy, Ethics, and Responsible AI
14. Testing Strategy
15. Twenty-Four-Hour Development Plan
16. Team Roles
17. Risks and Mitigation
18. Future Roadmap
19. Expected Deliverables
20. Final Product Definition

Important implementation note: Apple Vision detects visual landmarks; it does not independently translate arbitrary sign language. A suitable trained recognition model and validated vocabulary are required.

1. Executive Summary

SignaSense is an AI-powered iOS application designed to make communication more accessible between sign-language users and people who do not understand sign language. The application uses the iPhone camera to capture hand gestures and relevant facial cues, processes visual information to recognize supported signs, converts their interpreted meanings into text, and generates spoken output using native iOS speech capabilities.

Unlike a basic gesture-to-text application, SignaSense aims to incorporate facial context into its speech output. When the system recognizes a supported phrase and detects relevant expressive cues, it may select an appropriate speech style, such as neutral, warm, or emphatic.

The initial version focuses on a limited, validated vocabulary and a reliable end-to-end demonstration. Future versions may introduce continuous sign recognition, sentence-level interpretation, expanded vocabulary, improved expressive speech, and support for additional sign languages.

Product vision: Make everyday communication more inclusive by connecting signed gestures, written language, and spoken expression through accessible mobile technology.

2. Problem Statement

Communication between people who use sign language and people unfamiliar with it can be difficult, especially in everyday environments where an interpreter is unavailable. Existing communication methods may require manual typing, another person to interpret gestures, or separate applications for different needs. Text-only translation can also omit relevant visual context conveyed by facial expressions and other non-manual signals.

The problem is not simply the absence of voice output. A useful solution must recognize supported signs reliably, communicate uncertainty, protect users' privacy, and avoid changing the intended meaning of a signed message. SignaSense addresses this through a camera-based sign-to-text-and-speech experience with an additional layer for context-aware expressive output.

3. Product Objectives

3.1 Primary Objectives

- Recognize a defined set of sign-language gestures using the iPhone camera.
- Convert recognized signs into readable text.
- Provide spoken output using native iOS speech capabilities.
- Explore facial cues to select an appropriate expressive speech style.
- Deliver a simple, accessible interface that can be demonstrated within a 24-hour hackathon.
- Prefer on-device processing to reduce latency and minimize unnecessary transfer of sensitive visual data.

3.2 Secondary Objectives

- Create an architecture that supports future model improvements.
- Make uncertainty visible instead of silently displaying potentially incorrect translations.
- Allow users to control speech playback and expressive output.
- Establish a foundation for additional sign vocabulary and language support.

3.3 Success Metrics

- Recognition accuracy on a held-out test set of supported signs.
- End-to-end latency from a stable sign to displayed text.
- Correctness of text-to-speech output.
- Frequency of false detections during non-signing activity.
- Reliability under ordinary indoor lighting and camera conditions.
- Usability of the capture, correction, and playback workflow.

No accuracy or latency value should be advertised as achieved until measured on the actual prototype.

4. Target Users and Use Cases

4.1 Primary Users

- **Sign-language users:** People who wish to communicate selected signed messages to someone who does not understand their sign language.
- **Communication partners:** Family members, classmates, colleagues, service staff, and others who need to understand supported signs.

4.2 Secondary Users

- Accessibility advocates and educators.
- Developers researching mobile accessibility.
- Organizations exploring assistive communication tools.

4.3 Core Use Cases

- The user opens the app and selects the supported sign language.
- The user positions their hands in the camera view.
- The app detects the hand pose and analyzes a short sequence of frames.
- The recognition system predicts a supported sign.
- The app displays the recognized meaning and confidence state.
- The user can play the translated text as speech.
- When enabled and supported, facial cues help select a speech style.
- The user can correct an incorrect prediction or repeat the sign.
- If the system cannot recognize the sign confidently, it asks the user to try again.

5. Product Scope

5.1 MVP Features — Required

- Native iOS application built with Swift and Xcode.
- Rear-camera capture and camera permission/error handling.
- Hand-pose detection using Apple Vision.
- Recognition of a small, predefined set of signs using a compatible trained Core ML model or a clearly labelled prototype recognition pipeline.
- Mapping from recognized labels to text.
- Text-to-speech output using AVSpeechSynthesizer.
- Basic confidence threshold and uncertain-result handling.
- Simple interface for camera preview, recognized text, and playback.
- Basic testing on a physical iPhone where available.

5.2 MVP Features — If Time Permits

- Facial landmark analysis.
- A limited set of expressive speech presets.
- Sign stabilization across consecutive frames.
- Recognition history stored temporarily in memory.
- Manual text correction before speech playback.

- Basic onboarding and accessibility settings.

5.3 Out of Scope for the 24-Hour MVP

- Translation of every sign in ISL.
- Fully accurate, unrestricted sentence-level translation.
- General-purpose translation between multiple sign languages.
- Reliable inference of a person's actual emotional state.
- Natural, human-level emotional voice acting generated entirely on-device.
- Medical, legal, emergency, or other high-stakes translation.
- A custom large-scale sign-language model trained from scratch during the hackathon.
- Cloud accounts, user profiles, and server infrastructure unless specifically required.

6. Functional Requirements

FR-01: Camera Access

Request camera permission before access. Display a clear message if permission is denied and guidance for enabling it in iOS Settings. Indicate whether the camera is active.

FR-02: Visual Processing

Process captured frames to extract supported visual features. Use Vision hand-pose observations and optional face landmarks. Handle missing hands, partially visible gestures, and incomplete observations without crashing.

FR-03: Sign Recognition

Classify supported signs using a trained model or documented prototype method. Normalize landmarks, represent temporal movement where needed, and return a label with confidence information.

FR-04: Recognition Stabilization

Evaluate a short sequence of predictions and accept results only when sufficiently stable and above the configured threshold. Avoid repeated speech until a new signing sequence or explicit playback.

FR-05: Text Translation

Map recognized labels to predefined text meanings. Keep the mapping layer separate from the visual model. Use validated mappings rather than inventing complete sentences from isolated signs.

FR-06: Facial Cue Analysis

Optionally analyze facial landmarks or other relevant visual features. Produce a limited cue estimate and uncertainty state. Do not treat expression as definitive evidence of internal emotion.

FR-07: Expressive Speech Selection

Optionally select a supported speech preset such as neutral, warm, emphatic, or calm. Default to neutral when the estimate is uncertain. Do not change the recognized words or invent emotional meaning.

FR-08: Text-to-Speech

Use AVSpeechSynthesizer and AVSpeechUtterance. Allow play and stop. Handle repeated requests and interruptions. Voice parameters do not guarantee natural emotional delivery.

FR-09: User Interface

Display camera preview, camera status, recognition status, recognized text, uncertainty, play/stop controls, retry, and an expressive-speech toggle if implemented.

FR-10: Error Handling

Handle camera denial or failure, no visible hands, low-confidence predictions, unsupported signs, model loading failure, speech interruption, and missing or incompatible model assets. Provide actionable messages.

7. Detailed Technology Stack

Technology	Purpose and Implementation
Swift	Primary language for app state, camera workflow, recognition orchestration, translation mapping, speech output, and user interactions.
Xcode	Project creation, source/assets management, permission configuration, debugging, signing, simulator testing, and device deployment.
AVFoundation	Camera capture using AVCaptureSession, AVCaptureDeviceInput, and AVCaptureVideoDataOutput. Selected frames are forwarded to visual processing without blocking the UI thread.
Apple Vision	VNDetectHumanHandPoseRequest for hand-joint observations; VNDetectFaceLandmarksRequest and VNDetectFaceRectanglesRequest for optional facial analysis. Vision supplies observations, not sign-language translation.
Core ML	On-device inference for a compatible trained model. Inputs may be normalized landmarks or temporal sequences. A model must be obtained, trained, or converted; it is not supplied automatically.
AVSpeechSynthesizer	Native text-to-speech using AVSpeechUtterance. Supported settings may include rate, pitch multiplier, volume, and voice selection.
SwiftUI	Main interface for preview container, recognized text, status, buttons, and settings. UIViewRepresentable can bridge UIKit camera preview components.
Foundation	Core data types, collections, utilities, and resource handling.
Combine / Swift Concurrency	Coordination of asynchronous processing and UI updates; choose a consistent approach appropriate to the target iOS version.
XCTest	Unit and integration tests for translation mappings, recognition state, confidence filtering, and speech selection.
Xcode Instruments	Performance profiling, CPU and memory inspection, and identification of processing bottlenecks.
Git / GitHub	Version control, collaboration, code review, and tracking changes.

7.1 Backend Strategy

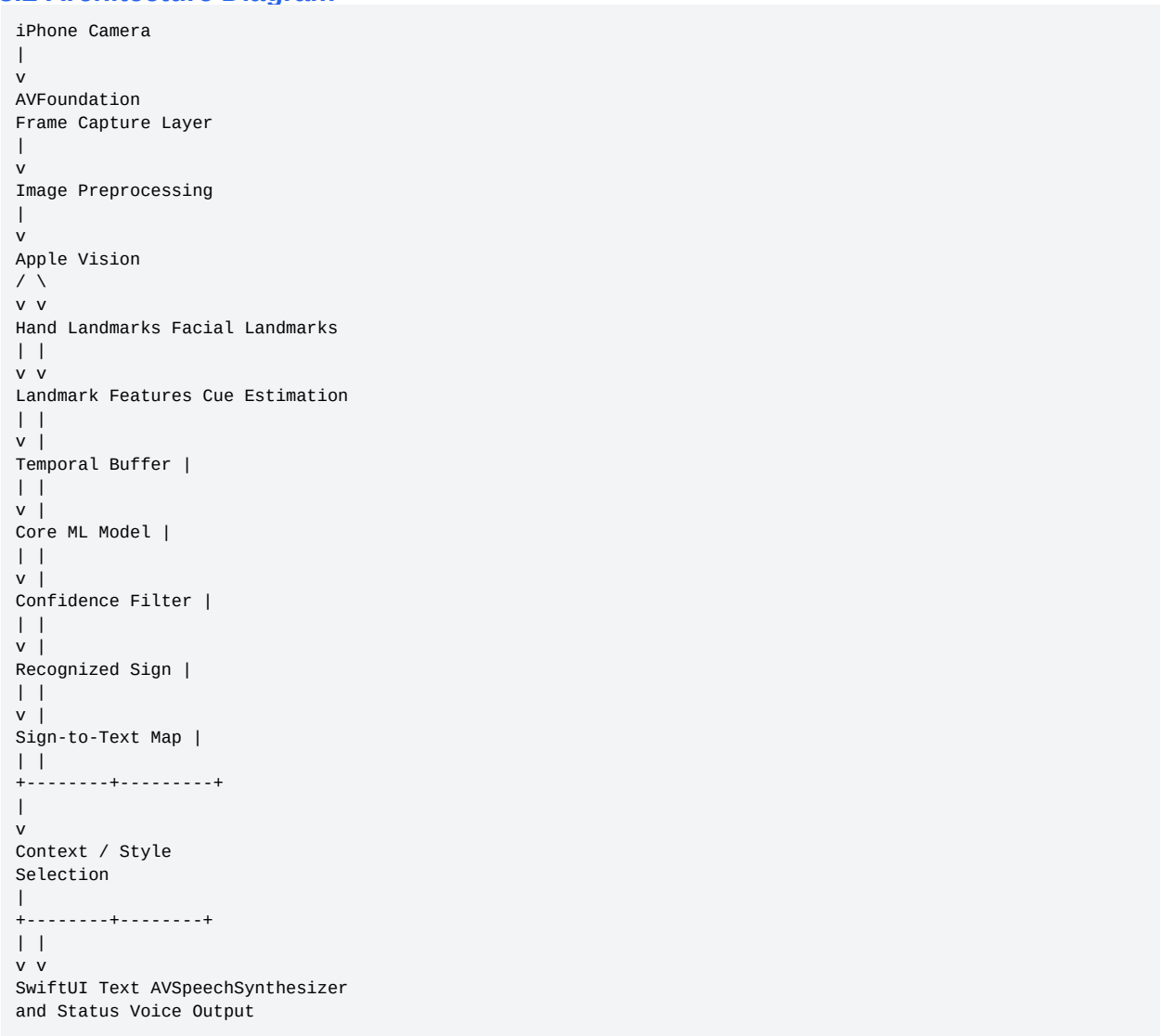
No backend is mandatory for the MVP. Camera processing, recognition, translation mapping, and speech output should run locally wherever feasible. A backend may be considered later for model updates, optional analytics, or synchronized preferences, with a clear purpose and appropriate consent.

8. System Architecture

8.1 High-Level Architecture

- **Input layer:** iPhone camera and AVFoundation capture session.
- **Visual processing layer:** Frame preprocessing, Vision hand-pose detection, and optional facial landmark detection.
- **Recognition layer:** Landmark normalization, temporal sequence buffer, Core ML classifier, confidence and stability filter.
- **Interpretation layer:** Recognized sign label, sign-to-text mapping, optional expression estimation, and speech-style selection.
- **Output layer:** SwiftUI interface, translated text, and AVSpeechSynthesizer.

8.2 Architecture Diagram



8.3 End-to-End Data Flow

- The user opens the app and grants camera permission.
- AVFoundation starts the capture session.
- Selected frames are sent to Vision.
- Vision returns available hand-pose observations.
- The recognition layer normalizes landmarks and maintains a short temporal window.
- The Core ML model predicts a supported sign.
- The stability filter determines whether the prediction is ready for display.
- The translation layer maps the label to text.
- If enabled, the expression layer evaluates relevant facial cues.
- A speech-style selector chooses a supported preset or defaults to neutral.
- SwiftUI displays the recognized text.
- AVSpeechSynthesizer speaks the text when requested.

8.4 Architectural Principles

- Modularity: major capabilities should be replaceable independently.
- Local-first processing: prefer on-device computation for responsiveness and privacy.

- Graceful degradation: basic text and neutral speech must work if expression analysis fails.
- Confidence awareness: uncertain predictions must not be presented as verified translations.
- Temporal awareness: use sequences for signs that depend on movement.
- Accessibility: keep controls clear, readable, and easy to operate.

9. Recommended Application Structure

```
SignaSense/
■■■ App/
■ ■■■ SignaSenseApp.swift
■■■ Models/
■ ■■■ SignPrediction.swift
■ ■■■ RecognitionState.swift
■ ■■■ SpeechStyle.swift
■■■ Views/
■ ■■■ HomeView.swift
■ ■■■ CameraPreviewView.swift
■ ■■■ TranslationView.swift
■ ■■■ SettingsView.swift
■■■ Services/
■ ■■■ CameraService.swift
■ ■■■ HandPoseService.swift
■ ■■■ FaceCueService.swift
■ ■■■ SignRecognitionService.swift
■ ■■■ TranslationService.swift
■ ■■■ SpeechService.swift
■■■ Resources/
■ ■■■ SignClassifier.mlmodel
■ ■■■ SignMappings.json
■■■ Tests/
■■■ TranslationServiceTests.swift
■■■ RecognitionLogicTests.swift
```

This is a proposed structure, not a requirement to create every file during the hackathon. Optional components can be omitted until the relevant feature is implemented.

10. Data and Model Requirements

10.1 Vocabulary Selection

The MVP should target a small, clearly documented vocabulary. Choose signs that can be distinguished reliably under the available camera conditions and demonstrated consistently. Validate the selected vocabulary with people knowledgeable in the target sign language wherever possible.

10.2 Dataset Requirements

- Different people signing the same supported sign.
- Different hand sizes, orientations, and signing speeds.
- Ordinary indoor lighting variations.
- Different camera distances and backgrounds.
- Negative examples such as resting hands and unrelated movements.
- Labels verified against the target sign language.
- A dataset suitable for the intended use and permitted for the proposed training and demonstration.

10.3 Training and Evaluation

If model training is necessary, perform it before the event or limit it to a small feasible experiment. Separate training and test examples by signer where possible; otherwise, evaluation may overestimate generalization to new users.

- Per-class precision and recall where feasible.
- Overall accuracy on a held-out test set.
- Confusion between visually similar signs.

- False detections during non-signing activity.
- Performance across available test conditions.

10.4 Model Fallback

If a validated model is unavailable, demonstrate the camera pipeline and a limited prototype classifier. Clearly describe this as a prototype rather than general-purpose sign-language recognition.

11. User Experience and Interface Requirements

11.1 Home Screen

- Project name and short explanation.
- Camera preview.
- Start or stop capture control.
- Recognition status.
- Recognized text.
- Play and stop speech buttons.
- Retry or clear-result action.

11.2 Recognition States

- Camera initializing.
- Ready to recognize.
- Detecting hands.
- Processing a gesture.
- Translation available.
- Unsupported or uncertain gesture.
- Camera or model error.

11.3 Accessibility

- Use readable text and sufficient contrast.
- Provide labels for interactive controls.
- Avoid communicating status through color alone.
- Support Dynamic Type where practical.
- Keep interaction simple and predictable.
- Avoid unnecessary flashing, animations, or audio playback without user control.

12. Non-Functional Requirements

Category	Requirement
Performance	Responsive enough for a live demonstration. Measure latency on the target device; avoid processing every frame if it causes dropped frames or excessive resource use.
Reliability	Do not crash when a hand leaves the frame, a face is not detected, or the model returns an uncertain result.
Privacy	Process locally wherever feasible. Do not record or upload raw video by default.
Maintainability	Keep camera capture, recognition, translation, expression analysis, and speech generation in separate modules.
Compatibility	Choose an iOS deployment target that supports the selected APIs and is compatible with the available Xcode version and test device.

13. Privacy, Ethics, and Responsible AI

- Obtain camera permission through the standard iOS permission mechanism.
- Explain why camera access is required.
- Avoid recording or uploading raw video by default.
- Keep processing local where feasible.
- Do not identify signers or infer identity.
- Do not claim that visible expressions reveal a person’s true emotions.
- Do not alter translation meaning based solely on an expression estimate.
- Provide uncertainty states and allow retry or correction.
- Test with diverse signers and camera conditions.
- Present the MVP as an assistive prototype, not a replacement for qualified interpreters in high-stakes settings.

14. Testing Strategy

14.1 Unit Tests

Test translation mapping, confidence threshold logic, speech-style selection, and recognition state transitions independently.

14.2 Integration Tests

Verify that camera frames reach the visual processing layer, predictions reach the translation layer, and translated text reaches the speech service.

14.3 Device Tests

Test on a physical iPhone where available, including camera permission handling, app backgrounding, speech interruption, and repeated signing attempts. The iOS Simulator can validate much of the interface and non-camera logic, but is not a substitute for real-camera and device-performance testing.

14.4 Recognition Tests

Evaluate supported signs using held-out examples, different signers, and non-signing movements. Document false positives and signs that cannot yet be distinguished reliably.

14.5 Acceptance Criteria

- The app launches successfully on the target iOS environment.
- Camera permission and denial paths work.
- The camera preview operates on a supported test device.
- The recognition pipeline returns results for the selected vocabulary.
- The confidence filter can reject uncertain predictions.
- Recognized labels map to intended text.
- The speech service reads displayed text aloud.
- Repeated predictions do not continuously trigger unwanted speech.
- Expression analysis, if included, can fail without breaking basic translation.
- The team can demonstrate a complete, tested gesture-to-text-and-speech workflow.

15. Twenty-Four-Hour Development Plan

Time	Tasks	Priority / Output
Hours 0–3	Finalize small vocabulary; confirm dataset/model availability; create Xcode project; configure permissions; assign roles.	Scope and project setup
Hours 3–7	Implement AVFoundation capture and preview; integrate Vision hand-pose detection; verify landmarks.	Working camera pipeline
Hours 7–12	Integrate model or prototype classifier; implement temporal buffering and confidence checks; build sign-to-text mapping.	Recognized labels and text
Hours 12–15	Integrate AVSpeechSynthesizer; add play/stop; handle repeated predictions and interruptions.	Speech output
Hours 15–19	Add facial landmark analysis if feasible; implement limited speech presets and neutral fallback.	Optional expressive output
Hours 19–22	Run end-to-end tests; fix integration errors; test uncertain signs; measure basic metrics.	Stabilized prototype
Hours 22–24	Prepare reliable test cases, genuine screenshots, demo sequence, and limitation notes.	Presentation-ready demo

Priority rule: If the schedule slips, preserve the camera-to-recognition-to-text-to-speech workflow first. Add facial-cue analysis and expressive speech only after the core pipeline works.

16. Team Roles

- **iOS Developer:** SwiftUI interface, camera integration, app lifecycle, and permissions.
- **Computer Vision / ML Developer:** Vision integration, landmark processing, model integration, and evaluation.
- **AI / Data Developer:** Dataset selection and validation, model training or conversion if needed, and confidence calibration.
- **Integration and Testing Lead:** End-to-end integration, test cases, error handling, and demo preparation.

Responsibilities may be combined depending on team size.

17. Risks and Mitigation

Risk	Potential Impact	Mitigation
No compatible trained model	Recognition may not work reliably.	Select vocabulary and model before implementation; use a clearly labelled fallback.
Static landmarks cannot distinguish a sign	Incorrect predictions.	Use temporal features and select distinguishable signs.
Facial cues are ambiguous	Inappropriate speech style.	Use conservative presets and neutral fallback.
Limited dataset diversity	Poor generalization.	Test with different signers and document limitations.
Camera performance issues	Slow or unstable demo.	Reduce processing frequency and avoid blocking the main thread.
Xcode or device setup problems	Lost development time.	Confirm access to a compatible Mac and physical iPhone early.
Unwanted repeated speech	Poor user experience.	Use recognition stabilization and explicit playback controls.
Incorrect translation	Miscommunication.	Use validated labels, confidence thresholds, and retry options.

18. Future Roadmap

Version 1 — Hackathon MVP

- Limited sign vocabulary.
- Camera-based recognition.
- Text output.
- Native speech output.
- Basic confidence handling.
- Optional expressive speech presets.

Version 2 — Improved Recognition

- Expanded vocabulary.
- Improved temporal recognition.
- More extensive signer-independent evaluation.
- Better handling of camera angles and lighting.
- Improved correction and recognition history.

Version 3 — Sentence-Level Interpretation

- Continuous signing sequences.
- Language-specific ordering and grammar.
- Better handling of two-handed and movement-dependent signs.
- Validated sentence-level translation.

Version 4 — Advanced Expressive Communication

- Improved contextual interpretation of non-manual linguistic signals.
- More natural expressive speech where technically and ethically appropriate.
- User-controlled voice styles.
- Additional sign-language support based on suitable datasets and validation.

19. Expected Deliverables

- A working Swift/Xcode iOS project.
- Camera preview and hand-pose detection.
- A small, documented supported vocabulary.
- A functional recognition and text-mapping pipeline.
- Text-to-speech output.
- Optional facial-cue and expressive speech functionality, if completed.
- Basic test results and known limitations.
- A short demonstration video or live demo.
- A concise technical explanation of the architecture.

20. Final Product Definition

SignaSense is a mobile accessibility prototype that explores how supported sign-language gestures can be converted into text and speech, with optional facial-cue-aware expressive output. Its core value is the integration of visual recognition, understandable translation output, and native iOS speech into a single experience. The first release will prioritize reliability, transparency, privacy, and a clearly defined vocabulary over unsupported claims of universal sign-language translation.

Product tagline: Turning gestures into words, connecting people through communication.

Guiding principle: Recognize carefully. Translate transparently. Speak accessibly.